# Week-1

Q1 WAP to simulate CPU scheduling algorithm of
1. FCFS
2. SJF

```
#include<stdio.h>
int main(){
    int n, i, choice;
    int at[10], bt[10], ct[10], tat[10], wt[10];
    printf("Enter no of process");
    scanf("%d", &n);
    for(i=0; i<n; i++)
        printf("Enter AT & BT for P%d :", i+1);
        scanf("%d%d", &at[i], &bt[i]);
}
printf("Select Type : \n");
printf("1 Non preemptive \n");
printf("2 : preemptive \n");
printf("Enter choice :");
scanf("%d", &choice);
if(choice == 1){
    ct[0] = at[0] + bt[0];
    for(i=1; i<n; i++){
        if(ct[i-1] < at[i]){
            ct[i] = at[i] + bt[i];
        }else
            ct[i] = ct[i-1] + bt[i];
    }
    for(i=0; i<n; i++){
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }
    printf("\nP \t AT \t BT \t CT \t TAT \t WT");
    for(i=0; i<n; i++){
        printf("P%d \t %d \t %d \t %d \t %d \t %d \t %d \t %d \t",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }
}
else printf("FCFS doesn't support preemptive \n");
}
```

```
avwt /= n;
avtat /= n;
printf("\n Average waiting time : %f", avwt);
printf("\n Average Turnaround time : %f \n", avtat);
return 0;
```

Output :-

```
Enter no of process : 2
Enter AT & BT for P1 : 0 5
Enter AT & BT for P2 : 0 4
Select Type :
  1. Non-preemptive
  2. Preemptive
Enter your choice : 1
```

| P | AT | BT | CT | TAT | WT |
|---|---|---|---|---|---|
| $P_1$ | 0 | 5 | 5 | 5 | 0 |
| $P_2$ | 0 | 4 | 9 | 9 | 5 |

Average waiting time : 2.500000
Average Turnaround time : 7.000000

```
#include<stdio.h>
#include<stdio.h>
int main()
{
    int n, i, time = 0, choice, done;
    int at[10], bt[10], ct[10], tat[10], wt[10], rt[10];
    printf("Enter no of process : ");
    scanf("%d", &n);
    for(i = 0; i < n; i++){
        printf("Enter AT & BT for P%d : ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i];
    }
    printf("Select type :\n");
    printf("1. Non preemptive\n");
    printf("2. Preemptive\n");
    printf("Enter your choice : ");
    scanf("%d", &choice);
    if(choice == 1){
        int vis[10] = {0};
        while(done < n){
            int min = INT_MAX, p = -1;
            for(int i = 0; i < n; i++){
                if(at[i] <= time && !vis[i]
                    && bt[i] < min){
                    min = bt[i];
                    p = i;
                }
            }
            if(p == -1){
                time++;
                continue;
            }
            time += bt[p];
            ct[p] = time;
            vis[p] = 1;
            done++;
        }
    }
}
```

```
else {
    while (done < n) {
        int min = INT_MAX, p = -1;
        for (i=0; i<n; i++) {
            if (at[i] <= time && rt[i] > 0 && rt[i] < min) {
                min = rt[i];
                p = i;
            }
        }
        if (p == -1) {
            time++;
            continue;
        }
        rt[p]--;
        time++;
        if (rt[p] == 0) {
            ct[p] = time;
            done++;
        }
    }
}
printf("\n P\t AT\t BT\t CT\t TAT\t WT\n");
for (i=0; i<n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    printf("P %d\t %d\t %d\t %d\t %d\t %d\n", i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}
printf("Average tat: %f", (tat - total)/n);
printf("Average wt: %f", (wt - total)/n);
return 0;
}
```

Output:

```
Enter no of processes: 2
AT & BT of P1: 0 5
AT & BT of P2: 1 3
AT & BT of P3: 2 4
Select type:
1. Non preemptive
2. preemptive
Enter choice: 1
```

| P | AT | BT | CT | TAT | WT |
|---|---|---|---|---|---|
| P1 | 0 | 5 | 5 | 5 | 0 |
| P2 | 1 | 3 | 8 | 7 | 4 |
| P3 | 2 | 4 | 12 | 10 | 6 |

Average tat: 7.333333 Average wt: 3.333333

ii)

```
Enter no of processes: 3
AT and BT of P1: 0 5
AT and BT of P2: 1 3
AT and BT of P3: 2 4
Select type:
1. Non preemptive
2. preemptive
Enter choice: 2
```

| P | AT | BT | CT | TAT | WT |
|---|---|---|---|---|---|
| P1 | 0 | 5 | 8 | 8 | 3 |
| P2 | 1 | 3 | 4 | 3 | 0 |
| P3 | 2 | 4 | 12 | 10 | 6 |

Average tat: 7.000000 Average wt: 3.000000

8/8

13/3

Week - 2

```
#include<stdio.h>
struct process {
    int at;
    int bt;
    int wt;
    int tat;
    int ct;
    int pr;
    int done;
    int rt;
};
void priority (struct process p[], int n) {
    int t=0, completed=0;
    float atat=0.0f, awt=0.0f;
    while (completed<n) {
        int idx=-1;
        int max_prior=9999;
        for (int i=0; i<n; i++) {
            if (p[i].at<=t && p[i].done==0) {
                if (p[i].pr<max_prior) {
                    max_prior=p[i].pr;
                    idx=i;
                }
                else if (p[i].pr==max_prior && idx==-1) {
                    if (p[i].at<p[idx].at) {
                        idx=i;
                    }
                }
            }
        }
    }
    if (idx!=-1) {
        p[idx].rt--;
        t++;
        if (p[idx].rt==0) {
            completed++;
            p[idx].ct=t;
            p[idx].tat=p[idx].ct-p[idx].at;
            p[idx].wt=p[idx].tat-p[idx].bt;
            p[idx].done=1;
            atat+=p[idx].tat;
            awt+=p[idx].wt;
        }
    }
    else {
        t++;
    }
    printf("%5s %5s %5s %5s %5s %5s %5s\n",
           pid, at, bt, ct, wt, tat, priority);
    for (int idx=0; idx<n; idx++) {
        printf("%5d %5d %5d %5d %5d %5d %5d\n",
               idx+1, p[idx].at, p[idx].bt, p[idx].ct, p[idx].
               p[idx].tat, p[idx].pr);
    }
    printf("Average waiting time: %.2f\n", awt/n);
    printf("Average turnaround time: %.2f\n", atat/n);
}
int main()
    int n;
    printf("Enter no of process: ");
    scanf("%d", &n);
    struct process p[n];
    for (int i=0; i<n; i++) {
        printf("Arrival time: ");
        scanf("%d", &p[i].at);
        printf("Burst time: ");
        scanf("%d", &p[i].bt);
        printf("priority:");
        scanf("%d", &p[i].pr);
        p[i].done=0;
```

```
p[i].rt = p[i].bt;
}
printProcess(p, n);
return 0;
}
```

Output :-

```
Enter no of process : 4
Arrival time : 0
Burst time : 2
Priority : 3
Arrival time : 2
Burst time : 1
Burst Prio : 2
Arrival time : 6
Burst time : 4
Priority : 4
Arrival time : 4
Burst time : 2
Priority : 1
```

| PID | AT | BT | CT | WT | TAT | Priority |
|---|---|---|---|---|---|---|
| 1 | 0 | 2 | 2 | 0 | 2 | 3 |
| 2 | 2 | 1 | 3 | 0 | 1 | 2 |
| 3 | 6 | 4 | 10 | 0 | 4 | 4 |
| 4 | 4 | 2 | 6 | 0 | 2 | 1 |

Avg waiting time : 0.00

Avg turnaround time : 2.25

Week-3

Q Write a C prog to simulate Round Robin

```
#include <stdio.h>
struct process {
    int at;
    int bt;
    int rt;
    int wt;
    int tat;
    int ct;
};
#define max 100
void RoundRobin (struct process p[], int n, int tq) {
    int t = 0, completed = 0;
    int queue[max], front = 0, rear = 0, visited[max] = {0};
    for (int i = 0; i < n; i++)
        p[i].rt = p[i].bt;
    for (int i = 0; i < n; i++) {
        if (p[i].at == 0) {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }
    if (rear == 0) {
        int min_at = p[0].at, idx = 0;
        for (int i = 1; i < n; i++) {
            if (p[i].at < min_at) {
                min_at = p[i].at;
                idx = i;
            }
        }
    }
}
```

```
    t = p[idx].at;
    queue[rear++] = idx;
    visited[idx] = 1;
}
while (completed < n) {
    int idx = queue[front++];
    if (p[idx].rt > tq) {
        t += tq;
        p[idx].rt -= tq;
    } else {
        t += p[idx].rt;
        p[idx].ct = t;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;
        p[idx].rt = 0;
        completed++;
    }
    for (int i = 0; i < n; i++) {
        if (i != idx && !visited[i] && p[i].at <= t) {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }
    if (p[idx].rt > 0) {
        queue[rear++] = idx;
    }
}
if (front == rear && completed < n) {
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            t = (t < p[i].at) ? p[i].at : t;
            queue[rear++] = i;
            visited[i] = 1;
            break;
        }
    }
}
printf("\nID\tAT\tBT\tCT\tTAT\tWT\n");
float total_wt = 0, total_tat = 0;
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
    i, p[i].at, p[i].bt, p[i].ct, p[i].wt, p[i].tat);
    total_wt += p[i].wt;
    total_tat += p[i].tat;
}
printf("\nAverage Waiting Time : %.2f\n", total_wt/n);
printf("Average Turnaround Time : %.2f\n", total_tat/n);
}
int main() {
    int n, tq;
    struct process p[max];
    printf("Enter no of process : ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Enter arrival time for process %d : ", i);
        scanf("%d", &p[i].at);
        printf("Enter burst time for process %d : ", i);
        scanf("%d", &p[i].bt);
    }
    printf("Enter time quantum : ");
    scanf("%d", &tq);
    RoundRobin(p, n, tq);
    return 0;
}
```

Output :-

Enter no of processes : 3
Enter arrival time : 2
Enter burst time : 5
Enter arrival time : 3
Enter burst time : 2
Enter arrival time : 5
Enter burst time : 1
Enter time Quantum : 3

| PID | AT | BT | CT | WT | TAT |
|---|---|---|---|---|---|
| 0 | 2 | 5 | 10 | 3 | 8 |
| 1 | 3 | 2 | 7 | 2 | 4 |
| 2 | 5 | 1 | 8 | 2 | 3 |

Avg waiting time : 2.33
Avg Turnaround time : 5.00

Enter no of proces. : 3
Enter arrival time : 2

Enter time quantum : 5

| PID | AT | BT | CT | WT | TAT |
|---|---|---|---|---|---|
| 0 | 2 | 5 | 7 | 0 | 5 |
| 1 | 3 | 2 | 9 | 4 | 6 |
| 2 | 5 | 1 | 10 | 4 | 5 |

Average waiting time : 2.67
Average Turnaround time : 5.33

# Higher the time quantum lower is the avg waiting and turnaround time

Q Write a C prog to simulate multilevel scheduling algorithm. Assume processes are divided in two categories system process & User process. System process have more priority than user process

```
#include <stdio.h>
#define MAX 100
typedef struct {
        int pid;
        int at, bt, rt;
        int ct, wt, tat;
        int type;
} Process;

typedef struct {
        int items[MAX];
        int front, rear;
} Queue;

void initQueue(Queue *q) {
        q->front = q->rear = -1;
}

int isEmpty(Queue *q) {
        return q->front == -1;
}

void enqueue(Queue *q, int value) {
        if (q->rear == MAX-1) return;
        if (q->front == -1) q->front = 0;
        q->items[++q->rear] = value;
}

int dequeue(Queue *q) {
        if (isEmpty(q)) return -1;
        int val = q->items[q->front];
        if (q->front == q->rear)
                q->front = q->rear = -1;
        else
                q->front++;
        return val;
}
```

```
int main() {
    int n, i, time=0, completed=0, index=0;
    Process p[MAX], temp;
    Queue systemQ, userQ;
    InitQueue(&systemQ);
    InitQueue(&userQ);
    printf("Enter no of processes: ");
    scanf("%d", &n);

    for(i=0; i<n; i++) {
        printf("\n Process %d \n", i);
        p[i].pid = i
        printf("Enter Arrival time: ");
        scanf("%d", &p[i].at);
        printf("Enter burst time: ");
        scanf("%d", &p[i].bt);
        printf("Enter type (0 = System, 1 = User): ");
        scanf("%d", &p[i].type);
        p[i].rt = p[i].bt;
    }

    for(i=0; i<n-1; i++) {
        for(int j=i+1; j<n; j++) {
            if(p[i].at > p[j].at) {
                temp=p[i];
                p[i]=p[j];
                p[j]=temp;
            }
        }
    }
```

```
    int current = -1;
    while(completed<n) {
        while(index<n && p[index].at <= time) {
            if(p[index].type == 0)
                enqueue(&systemQ, index);
            else
                enqueue(&userQ, index);
            index++;
        }
        if(current != -1) {
            if(p[current].type == 1 && !IsEmpty(&systemQ))
                enqueue(&userQ, current);
                current = -1;
            }
        }
        if(current == -1) {
            if(!IsEmpty(&systemQ))
                current = dequeue(&systemQ);
            else if(!IsEmpty(&userQ))
                current = dequeue(&userQ);
            else {
                time++;
                continue;
            }
        }
        p[current].rt--;
        time++;
        if(p[current].rt == 0) {
            p[current].ct = time;
            p[current].tat = p[current].ct - p[current].at;
            p[current].wt = p[current].tat - p[current].bt;
            completed++;
            current = -1;
        }
    }
```

```
float totalWT = 0, totalTAT = 0;
printf("\nID\tType\tAT\tBT\tCT\tWT\tTAT");
for(int pid=0; pid<n; pid++){
    for(i=0; i<n; i++){
        if(p[i].id == pid){
            printf("%d\t%s\t%d\t%d\t%d\t%d\t%d",
              p[i].id,
              (p[i].type==0)?"System":"User",
              p[i].at,
              p[i].bt,
              p[i].ct,
              p[i].wt,
              p[i].tat);
            totalWT += p[i].wt;
            totalTAT += p[i].tat;
        }
    }
}
printf("\nAverage Waiting Time: %.2f", totalWT/n);
printf("\nAverage Turnaround Time: %.2f\n", totalTAT/n);
return 0;
}
```

Output:-

```
Enter burst time: 5
Enter type (0=System, 1=User): 0
```

Output:

```
Enter no of process: 4
Process 0
Enter arrival time: 0
Enter burst time: 5
Enter type (0= System, 1=User): 0
Process 1
Enter arrival time: 2
Enter burst time: 3
Enter type (0=System 1=User): 1
Process 2
Enter arrival time: 1
Enter burst time: 4
Enter type (0= System 1=User): 0
Process 3
Enter arrival time: 3
Enter burst time: 2
Enter type (0: System, 1=User): 1
```

| ID | Type | AT | BT | CT | WT | TAT |
|---|---|---|---|---|---|---|
| 0 | System | 0 | 5 | 5 | 0 | 5 |
| 1 | User | 2 | 3 | 12 | 7 | 10 |
| 2 | System | 1 | 4 | 9 | 4 | 8 |
| 3 | User | 3 | 2 | 14 | 9 | 11 |

```
Average Waiting Time: 5.00
Average Turn Around Time: 8.50
```

~~# Highest Time Qu~~

Week - 4

Write a C program to simulate Real time CPU scheduling algorithms

(a) Rate - Monotonic
(b) Earliest - deadline first
(c) Proportional scheduling

```
#include <stdio.h>
#include <math.h>
#define MAX 10
typedef struct {
    int id;
    int bt;
    int deadline;
    int period
    int ct, wt, tat
} process;
Process P[MAX], temp;
int n;
--- CPU Utilization ---
float calculateUtilization EDF() {
    float util=0;
    for ( int P=0; P<n; P++) {
    util += (float) p[P].bt/p[P].deadline;
    }
    return util;
}
float calculateUtilizationRMS()
    float util =0;
    for (int P=0; P<n; P++) {
      util += ((float)p[P].bt/p[P].period;
    }
    return util;
}
```

```
-- EDF ...
void edfScheduling() {
    for (int P=0; P<n-1; P++) {
        for (int j = P+1; j<n; j++) {
            if (p[P].deadline > p[j].deadline) {
                temp = p[P]
                p[P] = p[j]
                p[j] = temp;
            }
        }
    }
    int time=0;
    for (int P=0; P<n; P++) {
        time += p[P].bt
        p[P].ct = time
        p[P].tat = p[P].ct;
        p[P].wt = p[P].tat - p[P].bt;
    }
    int time = 0;
    for (int P=0; P<n; P++) {
        time += p[P].bt;
        p[P].ct = time;
        p[P].tat = p[P].ct;
        p[P].wt = p[P].tat - p[P].bt;
    }
float util = calculateUtilizationEDF()
printf ("\n ... Earliest Deadline First (EDF) Sch
... \n");
printf("\n CPU Utilization : %.2f \n", util);
if (util<=1)
    printf ("\n Schedulable (Utilization = 1) \n");
else
    printf ("\n Not Schedulable \n");
printf ("\nID BT Deadline CT WT TAT \n");
```

```
for (int i=0; i<n; i++){
    printf ("%d %d %d %d %d %d \n",
    p[i].id, p[i].bt, p[i].deadline,
    p[i].ct, p[i].wt, p[i].tat);
}
}
void rms_scheduling () {
    //sort by period
    for (int i=0; i<n-1; i++){
        for (int j=i+1; j<n; j++){
            if (p[i].period > p[j].period){
                temp = p[i]
                p[i]= p[j];
                p[j]=temp;
            }
        }
    }
    int time=0;
    for (int i=0; i<n; i++) {
        time + = p[i].bt;
        p[i].ct = time;
        p[i].tat= p[i].ct;
        p[i].wt = p[i].tat - p[i].bt;
    }
    float util= calculate Utilization when RMS();
    float bound = n* (pow(2,(1.0/n))-1);
    printf("\n === Rate Monotonic Scheduling (RMS)
        ---\n");
    printf("CPU Utilization : %2f \n", util);
    printf("RM Bound : %4f \n", bound);
    if (util <= bound)
        printf("Schedulable (Utilization <= RM Bound) \n")
    else
        printf("Not Schedulable \n");
```

```
printf("ID BT Period CT WT TAT\n");
for (int i=0; i<n; i++){
    printf("%d %d %d %d %d %d \n",
        p[i].id, p[i].bt, p[i].period,
        p[i].ct, p[i].wt, p[i].tat);
}
}
// --- MAIN ---
int main() {
    printf("Enter no of process: ");
    scanf("%d", &n);
    printf("\n Enter process details : \n");
    for (int i=0; i<n; i++){
        p[i].id =i;
        printf("\n Process %d : \n", i);
        printf("Burst Time: ");
        scanf("%d", &p[i].bt);
        printf("Deadline (for EDF): ");
        scanf("%d", &p[i].deadline);
        printf("Deadline (for RMS): ");
        scanf("%d", &p[i].period);
    }
    edf_scheduling ();
    rms_scheduling ();
    return 0;
}
```

$n(2^{1/n} - 1)$

~~ENTER NO~~

Output :-

Enter no of processes : 3
Enter process details
process 0:
Burst time : 3
Deadline (for EDF) : 7
Period (for RMS) : 20

process 1 :
Burst time : 2
Deadline (for EDF) : 7
Period (for RMS) : 5

process 2 :
Burst time : 2
Deadline (for EDF) : 8
Period (for RMS) : 10

- EDF Scheduling (Earliest Deadline First)..
- CPU Utilization is
Schedulable

| ID | BT | Deadline | CT | WT | TAT |
|---|---|---|---|---|---|
| 1 | 2 | 4 | 2 | 0 | 2 |
| 0 | 3 | 7 | 5 | 2 | 5 |
| 2 | 2 | 8 | 7 | 5 | 7 |

Rate Monotonic Scheduling (RMS)..
CPU Utilization : 0.75
RM Bound : 0.7798
Schedulable (Utilization <= RM Bound)

| ID | BT | Period | CT | WT | TAT |
|---|---|---|---|---|---|
| 1 | 2 | 5 | 2 | 0 | 2 |
| 2 | 2 | 10 | 4 | 2 | 4 |
| 0 | 3 | 20 | 7 | 4 | 7 |

- Proportional ~~Fair~~ Scheduling...
Total weight = 6

| ID | weight | CPU share |
|---|---|---|
| 0 | 2 | 0.33 |
| 1 | 1 | 0.17 |
| 2 | 3 | 0.50 |

Str
Poly.

24/4/26

## Bounded Buffer

1. Write a C program to simulate producer - consumer problem using semaphore

```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
#include<semaphore.h>
#include<unistd.h>
#define BUFFER_SIZE 3
int buffer[BUFFER_SIZE];
int in=0, out=0;
void producer (void *arg) {
        int item;
        while(1){
          item = rand()%100;
          wait(empty);
          wait(mutex);
          buffer[in]=item;
          printf("\n Produced %d at %d \n", item, in);
          in =(in+1) % Buffer-size;
          signal (mutex);
          signal(full);
          sleep(1);
        }
}
void* consumer (void *arg){
        int item
        while (1) {
          wait(full);
          wait(mutex);
          item= buffer[out];
          printf("\n Consumed %d from %d\n", item, out);
          out = (out+1) % BUFFER_SIZE;
          signal(mutex);
          signal(empty);
          sleep(1)
}
```

```
int main()
    pthread_t, P, C;
    sem_init(&empty, 0, Buffer_size);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);
    pthread_create(&P, NULL, producer, NULL);
    pthread_join(P, NULL);
    pthread_join(C, NULL);
    return 0;
}
```

Output :-

```
produced : 1 at 0
Consumed : 1 from 0
Produced : 7 at 1
Consumed : 7 from 1
Produced : 4 at 2
Consumed : 4 from 2
Produced : 0 at 3
Consumed : 0 from 3
Produced : 9 at 0
Consumed : 0 from 3
Produced : 4 at 1
Consume : 9 from 0
produced : 8 at 2
Consumed : 4 from 1
produced 8 at 3
Consumed 8 from 2
Produced 2 at 0
```

| Operation | Buffer state | in | out | empty | Full | mutex |
|---|---|---|---|---|---|---|
| Produce item 1 | [1, _, _] | 1 | 0 | 3 | 1 | 1 |
| consume/remove 1 | [_, _, _] | 1 | 1 | 4 | 0 | 1 |
| Produce 7 | [_ 7 _ _] | 2 | 1 | 3 | 1 | 1 |
| Remove 7 | [_ _ _ _] | 2 | 1 | 4 | 0 | 1 |
| Produce 4 | [_ _ 4 _] | 3 | 2 | 3 | 1 | 1 |
| Remove 4 | [_ _ _ _] | 3 | 3 | 4 | 0 | 1 |
| Produce 0 | [_ _ 0] | 0 | 3 | 3 | 1 | 1 |
| Produce 9 | [9 _ _ 0] | 1 | 3 | 2 | 2 | 1 |
| remove 0 | [9 _ _ _] | 1 | 0 | 3 | 1 | 1 |

Dining philosopher :

```
#include<stdio.h>
#include<pthread.h>
#include<semaphore.h>
#include<unistd.h>
#define N 5
sem_t forks[N];
sem_t mutex;
#define wait(x) sem_wait(&x)
#define signal(x) sem_post(&x)
void *philosopher(void *num){
    int id = *(int*)num;
    while(1){
        printf("philosopher %d is thinking\n", id);
        sleep(4);
        wait(mutex);
        wait(forks[id]);
        wait(forks[(id+1)%N]);
        printf("philosopher %d is eating\n", id);
        sleep(6);
        signal(forks[id]);
        signal(forks[(id+1)%N]);
    }
}
int main(){
    pthread_t ph[N];
    int ids[N];
    sem_init(&mutex,0,1);
    for(int i=0;i<N;i++)
        sem_init(&forks[i],0,1);
    for(int i=0;i<N;i++){
        id[i]=i;
        pthread_create(&ph[i],NULL,philosopher,&id[i]);
    }
    for(int i=0;i<N;i++)
        pthread_join(ph[i],NULL);
    return 0;
}
```

Output

```
philosopher 0 is thinking
philosopher 1 is thinking
philosopher 2 is thinking
philosopher 3 is thinking
philosopher 4 is thinking
philosopher 3 is eating
philosopher 3 is thinking
philosopher 4 is eating
philosopher 4 is thinking
philosopher 2 is eating
philosopher 1 is eating
```

| philosopher | state change | forks used |
|---|---|---|
| P0 | thinking | — |
| P1 | thinking | — |
| P2 | thinking | — |
| P3 | thinking | — |
| P4 | thinking | — |
| P3 | eating | F3 F4 |
| P3 | thinking | Released F3 F4 |
| P4 | ECL | F4, F0 |
| P4 | think | Release F4, F0 |

1. Write a program to simulate
   (i) Banker's problem
   (ii) Deadlock Detection

```
#include <stdio.h>
int main(){
    int n, m, i, j, k;
    printf("Enter no of process: ");
    scanf("%d", &n);
    printf("Enter no of resources: ");
    scanf("%d", &m);
    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];
    int finish[n], safeseq[n];
    printf("\nEnter Allocation Matrix:\n");
    for(i=0; i<n; i++){
        for(j=0; j<m; j++){
            scanf("%d", &alloc[i][j]);
        }
    }
    printf("\nEnter Maximum Matrix:\n");
    for(i=0; i<n; i++){
        for(j=0; j<m; j++){
            scanf("%d", &max[i][j]);
        }
    }
    printf("\nEnter Available Resources:\n");
    for(i=0; i<n; i++){
        for(j=0; j<m; j++){
            scanf("%d", &max[i][j]);
        }
    }
}
```

```
printf
for(i=0; i<n; i++){
    for(j=0; j<m; j++){
        need[i][j] = max[i][j] - alloc[i][j];
    }
}
for(i=0; i<n; i++){
    finish[i] = 0;
}
int count = 0;
while(count < n){
    int found = 0;
    for(i=0; i<n; i++){
        if(finish[i] == 0){
            for(j=0; j<m; j++){
                if(need[i][j] > avail[j])
                    break;
            }
            if(j == m){
                for(k=0; k<m; k++){
                    avail[k] += alloc[i][k];
                }
                safeseq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
    if(found == 0){
        printf("\nSystem is NOT in safe state.\n");
        return 0;
    }
}
```

```
printf("\n System is in SAFE state. \n Safe Sequence :\n");
for(i=0;i<n;i++){
        printf("P%d", safeseq[i]);
        if(i!=n-1)
          printf("->");
    }
    printf("\n");
    return 0;
}
else if(choice==2){
    printf("\n Enter Request Matrix: \n");
    for(i=0;i<n;i++)
      for(j=0;j<m;j++)
            scanf("%d",&request[i][j]);

    for(i=0;i<n;i++){
          finish[i]=1;
          for(j=0;j<m;j++)
            if(alloc[i][j]!=0)
              finish[i]=0;
    }
    int found;
    do{
      found=0;
      for(i=0;i<n;i++){
        if(finish[i]){
            for(j=0;j<m;j++)
              if(request[i][j]>avail[j])
                break;
            if(j==m){
              for(k=0;k<m;k++)
                avail[k]+=alloc[i][k];
              finish[i]=1;
              found=1;
            }
        }
      }while(found);
      found=0;
      printf("\n processes in deadlock: \n");
      for(i=0;i<n;i++){
          if(!finish[i]){
              printf("P%d ",i);
              found=1;
          }
      }
      if(!found)
          printf("No deadlock Detected");
      else
          printf("\n Invalid choice");
      return 0;
}
```

Output:-

```
Enter no of processes: 5
Enter no of resources: 3
Enter allocation matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
```

Enter max matrix:

7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter available Resources:

0 0 0

1. Safety Algorithm
2. ~~Deadlock Algorithm~~ Resource Request

Enter choice 2

Enter Request Matrix:

0 0 0
2 0 2
0 0 0
1 0 1
0 0 2

Process Pn Deadlock:

No deadlock detected

1. Safety Algorithm
2. Request Resource

Enter choice: 1

System P1 Pn a safe state

Safe sequence: - P1 → P3 → P4 → P0 → P2

## Deadlock detection

```
Pnt main ()
  Pnt n, m;
  Printf ("Enter no of processes:");
  Scanf ("%d", &n);
  Printf ("Enter no of resource type:");
  Scanf ("%d", &m);
  Pnt alloc[n][m], request[n][m];
  Pnt avail[m];
  Pnt finish[n];
   Printf ("\n Enter Allocation Matrix:\n");
   For (i=0; i<n; i++) {
          for (j=0; j<m; j++) {
              Scanf ("%d", &alloc[i][j]);
      }
   }
   Printf ("Enter request Matrix: \n");
   for (i=0; i<n; i++) {
      for (j=0; j<m; j++) {
          Scanf ("%d", &request[i][j]);
   }
}

Printf ("Enter Available Resources: \n");
 for (j=0; j<m; j++) {
      Scanf ("%d", &avail[j]);
}
 for (i=0; i<n; i++) {
     Pnt flag=0;
     for (j=0; j<m; j++) {
        if (alloc[i][j] != 0) {
            flag=1;
            break;
        }
     }
     if (flag==0)
         finish[i]=1;
} else finish[i]=0;
```

```
int found;
    do {
      found = 0;
      for (i=0; i<n; i++) {
        if (finish[i] == 0) {
            for (j=0; j<m; j++) {
                if (request[i][j] > avail[j])
                break;
            }
            if (j == m) {
                for (k=0; k<m; k++)
                    avail[k] += alloc[i][k];
            }
        }
    }
    } while (found);
    int deadlock = 0;
    printf ("\n Process in deadlock \n");
    for (i=0; i<n; i++) {
        if (finish[i] == 0) {
            printf ("P %d", i);
            deadlock = 1;
        }
    }
    if (deadlock == 0)
        printf ("\n No deadlock detected");
    printf ("\n");
    return 0;
}
```

Output - Case 1

```
Enter no of process : 5
Enter no of resource type : 3
Enter Allocation Matrix :
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Request Matrix :-
0 0 0
2 0 2
0 0 0
1 0 1
0 0 2
Enter available resource :
0 0 0
Process in deadlock
No deadlock detected
```

Case 2

```
Enter no of process : 5
Enter no of resource : 3
Enter allocation matrix :
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter request matrix :
0 2 0
2 0 2
0 0 2
1 0 1
0 0 2
```

Enter available resources:

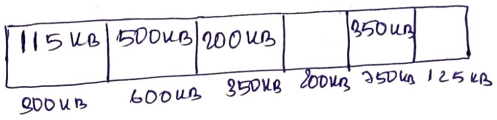
0 0 0

process in deadlock

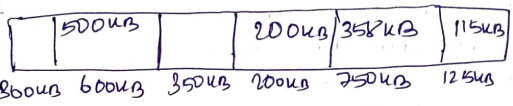
P0 P1 P2 P3 P4

WEEK-7

Q Given 6 memory partitions of 300KB, 600KB, 350KB, 200KB, 750KB, 125KB (in order). How would the first fit, best fit and worst fit algorithm place processes of size 115KB, 500KB, 358KB, 200KB, 375KB (in order)

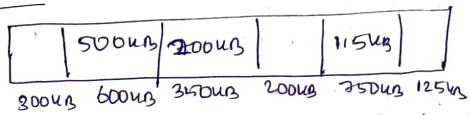



Best fit




375KB process is not allocated

Worst fit




process with size 358KB & 375KB cannot be allocated with memory

Q ~~Write~~ Write a C program simulate the following contiguous memory allocation techniques:

(a) worst fit
(b) best fit
(c) first fit

```
#include<stdio.h>
#define MAX 25
void firstfit(int blocksize[], int m, int processsize[], int n)
{
    int allocation[MAX];
    int temp[MAX];
    for(int i=0; i<m; i++)
        temp[i] = blocksize[i];
    for(int i=0; i<n; i++)
        allocation[i] = -1;
}
```

```
for (int i=0; i<n; i++) {
    for (int j=0; j<m; j++) {
        if (temp[j] >= processSize[i]) {
            allocation[i] = j;
            temp[j] -= processSize[i];
            break;
        }
    }
}

printf("\n--- First Fit ---\n");
printf("ProcessNo \t Process Size \t Block No.\n");
for (int i=0; i<n; i++) {
    // printf("%d/n", allocation[i]+1);
    printf("%d \t\t %d \t\t", i+1, processSize[i]);
    if (allocation[i] != -1)
        printf("%d \n", allocation[i]+1);
    else
        printf("Not Allocated");
}
}

void bestFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[MAX];
    int temp[MAX];
    for (int i=0; i<m; i++)
        temp[i] = blockSize[i];
    for (int i=0; i<n; i++) {
        allocation[i] = -1;
        for (int i=0; i<n; i++) {
            int bestIdx = -1;
            for (int j=0; j<m; j++) {
                if (temp[j] >= processSize[i]) {
                    if (bestIdx == -1 || temp[j] < temp[bestIdx])
                        bestIdx = j;
                }
            }
            if (bestIdx != -1) {
                allocation[i] = bestIdx;
                temp[bestIdx] -= processSize[i];
            }
        }
    }
```

```
printf("\n--- Best Fit ---\n");
printf("ProcessNo \t Process Size \t Block No.\n");
for (int i=0; i<n; i++) {
    printf("%d \t\t %d \t\t", i+1, processSize[i]);
    if (allocation[i] != -1)
        printf("%d \n", allocation[i]+1);
    else
        printf("Not Allocated\n");
}
}

void worstFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[MAX];
    int temp[MAX];
    for (int i=0; i<m; i++)
        temp[i] = blockSize[i];
    for (int i=0; i<n; i++)
        allocation[i] = -1;
    for (int i=0; i<n; i++) {
        int worstIdx = -1;
        for (int j=0; j<m; j++) {
            if (temp[j] >= processSize[i]) {
                if (worstIdx == -1 || temp[j] > temp[worstIdx])
                    worstIdx = j;
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            temp[worstIdx] -= processSize[i];
        }
    }
printf("\n--- Worst Fit ---\n");
printf("ProcessNo \t Process Size \t Block No.\n");
for (int i=0; i<n; i++) {
    printf("%d \t\t %d \t\t", i+1, processSize[i]);
    if (allocation[i] != -1)
        printf("%d \n", allocation[i]+1);
    else printf("Not Allocated\n");
}
```

```
int main(){
    int blocksize[MAX], processize[MAX];
    int m, n;
    printf("Enter number of memory blocks:");
    scanf("%d", &m);
    printf("Enter size of %d memory block: \n", m);
    for(int i=0; i<m; i++){
            scanf("%d", &blocksize[i]);
    }
    printf("Enter no of processes:");
    scanf("%d", &n);
    printf("Enter size of %d processes: \n", n);
        for(int i=0; i<n; i++){
        scanf("%d", &processize[i]);
    }
    firstfit(blocksize, m, processize, n);
    bestfit(blocksize, m, processize, n);
    coonsfit(blocksize, m, processize, n);
    return 0;
}
```

Output :-

Enter no of memory block : 6
Enter size of 6 memory block :
300
600
350
200
750
125
Enter no of process : 5
Enter size of 5 process : 115 500 358 200 375

- First fit :-

| Process No | process size | Block No |
|---|---|---|
| 1 | 115 | 1 |
| 2 | 500 | 2 |
| 3 | 358 | 5 |
| 4 | 200 | 3 |
| 5 | 375 | Not allocated |

BEST FIT

| process No | Size | Block No |
|---|---|---|
| 1 | 115 | 6 |
| 2 | 500 | 2 |
| 3 | 358 | 5 |
| 4 | 200 | 3 |
| 5 | 375 | Not allocated |

COONS FIT

| Process No | Size | Block No |
|---|---|---|
| 1 | 115 | 5 |
| 2 | 500 | 2 |
| 3 | 358 | Not Allocated |
| 4 | 200 | 3 |
| 5 | 375 | Not Allocated |

22/12/22

Q) Write a C program to implement following page replacement techniques (1) FIFO
(2) LRU
(3) Optimal

```
#include<stdio.h>
#include<conio.h>
void FIFO(int page[], int n, int f){
    int frame[10], i, j, k=0, found, fault=0;
    for(i=0; i<f; i++)
        frame[i]=-1;
    printf("\n-- FIFO page Replacement --\n");
    for(i=0; i<n; i++){
        found=0
        for(j=0; j<f; j++)
            if(frame[j]==page[i]){
                found=1;
                break;
            }
        if(!found){
            frame[k]=page[i];
            k=(k+1)%f;
            fault++;
        }
        printf("%d->", page[i]);
        for(j=0; j<f; j++)
            printf("%d ", frame[j]);
        printf("\n");
    }
    printf("Total Page Faults = %d\n", fault);
}
void LRU(int page[], int n, int f){
    int frame[10], time[10];
    int i, j, pos, fault=0, found;
    int counter=0, least;
    for(i=0; i<n; i++){
        found=0;
        for(j=0; j<f; j++){
            if(frame[j]==page[i]){
                counter++;
                time[j]=counter;
                found=1;
                break;
            }
        }
        if(!found){
            least=time[0];
            pos=0;
            for(j=0; j<f; j++){
                if(frame[j]==-1){
                    pos=j;
                    break;
                }
                if(time[j]<least){
                    least=time[j];
                    pos=j;
                }
            }
            counter++;
            frame[pos]=page[i];
            time[pos]=counter;
            fault++;
        }
        printf("%d->", page[i]);
        for(j=0; j<f; j++)
            printf("%d ", frame[j]);
        printf("\n");
    }
    printf("Total page faults = %d\n", fault);
}
```

```
void optimal (int pages[], int n, int f) {
        int frame[10];
        int p, i, k, pos, fault;
        int fault = 0, found;
        for (i=0; i<f; i++)
                frame[i] = -1;
        printf ("\n-- optimal Page Replacement --\n");
        for (i=0; i<n; i++) {
                found = 0;
                break;
        }
    }
    if (!found) {
        int replace = -1;
        fault = i+1;
        for (j=0; j<f; j++) {
            int flag = 0;
            for (k=i+1; k<n; k++) {
                if (frame[j] == pages[k]) {
                    if (k>fault) {
                        fault = k;
                        replace = j;
                    }
                    flag = 1;
                    break;
                }
            }
            if (flag == 0) {
                replace = j;
                break;
            }
        }
        if (replace == -1)
            replace = 0;
        frame[replace] = pages[i];
    } fault++;
```

```
    printf ("%d -> ", pages[i]);
    for (j=0; j<f; j++)
        printf ("%d", frame[j]);
    printf ("\n");
}
printf ("Total page Faults = %d\n", fault);
}

int main () {
    int pages[50], n, f, i, choice;
    printf ("Enter no of pages: ");
    scanf ("%d", &n);
    printf ("Enter page reference string: ");
    for (i=0; i<n; i++)
        scanf ("%d", &pages[i]);
    printf ("Enter no of frames: ");
    scanf ("%d", &f);
    printf ("\nChoose Algorithm");
    printf ("\n1. FIFO");
    printf ("\n2. LRU");
    printf ("\n3. Optimal");
    printf ("\nEnter your choice: ");
    scanf ("%d", &choice);
    switch (choice) {
        case 1:
            fifo (pages, n, f);
            break;
        case 2:
            LRU (pages, n, f);
            break;
        case 3:
            optimal (pages, n, f);
            break;
        default:
            printf ("Invalid choice\n");
    }
    return 0;
}
```

Output :-

Enter number of page : 10

Enter page reference string: 7 1 3 2 2 1 0 5 3 5

Enter no of frames 3

Choose Algorithm

1. FIFO
2. LRU
3. Optimal

Enter your choice :

FIFO Page replacement..

7 → 7 -1 -1
1 → 7 1 -1
3 → 7 1 3
2 → 2 1 3
2 → 2 1 3
1 → 2 1 3
0 → 2 0 3
~~0~~
5 → 2 0 5
3 → 3 0 5
5 → 3 0 5

Total page faults = 7

Choose Algorithm 1 FIFO 2. LRU 3. Optimal

LRU Page Replacement

7 → 7 -1 -1
1 → 7 1 -1
3 → 7 1 3
2 → 2 1 3
2 → 2 1 3
1 → 2 1 3
0 → 2 1 0
5 → 5 1 0
~~3 → 5 1 0~~

3 → 5 3 0
5 → 5 3 0

Total page faults = 7

Optimal Page Replacement..

7 → 7 -1 -1
1 → 1 -1 -1
3 → 1 3 -1
2 → 1 3 2
~~2~~ 2 → 1 3 2
1 → 1 3 2
0 → 0 3 2
5 → 5 3 2
5 → 5 3 2

Total page faults = 6

8 8 9
2 1 5 1 2 6